

Heterogeneous Reflex Reasoning: Multiple Symbolic Coprocessors with Typed Micro-State

Working Draft

August 2026

Abstract

Paper 2 asks whether strict automatic assistance should extend from one calculator to heterogeneous symbolic engines and persistent typed micro-state. We implement the gated substrate: Paper 1’s bounded calculator, a finite Horn forward-chaining engine, an ISA/frame graph engine, a strict typed-event router, append-only deduplicated state, derivation dependencies, deterministic mixed tasks, and machine-readable traces. A 42-example scripted smoke test produces 294 condition rows and 1,180 trace records. Heterogeneous, explicit, and oracle routes are perfect by construction; each single engine solves its own third; six controls cause no intervention. These results validate plumbing only and are marked `empirical: false`. Paper 1 did not pass its evidence gate, so no language-model experiment or heterogeneous-engine benefit is claimed.

1 Status and Evidence Gate

This paper is an implemented protocol, not an empirical result. Paper 1’s one-checkpoint pilot found that reflex calculator assistance tied the unassisted model and did not dominate explicit invocation. Its required multi-seed, multi-size replication has not run. The series gate for heterogeneous compute is therefore closed. We implement and test the deterministic substrate now so a future experiment can be frozen before model outcomes are observed; the smoke summary cannot be cited as language-model evidence.

2 Hypotheses for a Future Empirical Run

H1: heterogeneous reliability. On tasks requiring arithmetic, Horn closure, and graph inheritance, correctly routed heterogeneous assistance improves final accuracy over the same model, matched prompting, every single-engine condition, and explicit tools.

H2: scaling. As derivation depth and distractor count grow, heterogeneous assistance degrades more slowly than model-only and homogeneous baselines under matched engine, token, call, and latency accounting.

H3: typed-state reuse. Persistent derived facts reduce repeated generation or execution when later queries reuse prior premises, without stale or irrelevant state offsetting the gain.

Oracle selection estimates engine headroom. None of these hypotheses is tested by the scripted smoke run.

3 Implemented Architecture

The shared request envelope from `ccpu.common` carries request and candidate IDs, family, operation, target engine, typed payload, confidence, budget, and metadata. Strict events use one of three anchored forms:

```
[calculator] 1 + 2 + 3
[horn] { ... typed facts, rules, and query ... }
[graph] { ... typed ISA/frame data and query ... }
```

Tags embedded in prose, quoted tags, unknown families, malformed JSON, and events beyond 4,096 characters do not execute. The routing path is

DETECT → NORMALIZE → ROUTE → EXECUTE → PERSIST → TRACE.

3.1 Bounded calculator

The calculator is retained unchanged from Paper 1. Its allowlisted integer AST normalizes to postfix micro-IR and executes with limits on characters, nodes, depth, literal digits, exponent, value bits, stack size, and wall time. Generated code is never executed.

3.2 Finite Horn engine

An atom is a predicate with one to four identifier terms; variables begin with `?`. A rule has one head and one to eight body atoms. Facts and queries must be ground. The engine performs monotone forward chaining with defaults of 256 facts, 64 rules, and 16 closure rounds. New facts are deduplicated and persisted. Every derived fact records the state IDs of the premises used in its derivation. This is a finite Datalog-like Horn fragment, not unrestricted FOL or theorem proving.

3.3 ISA/frame graph

The graph engine stores child–parent ISA edges and entity–slot–value frames. It computes bounded transitive ISA closure and resolves a frame first on the entity, then on ancestors. Multiple values at the selected specificity are **ambiguous**; missing values are **unknown**. Defaults bound closure to 512 edges and depth 16.

3.4 Persistent typed state

State is append-only and deduplicated by a canonical content fingerprint. Each item stores an ID, kind, typed payload, dependency IDs, and provenance, with a default 512-item budget. Calculator results, graph assertions, Horn facts, derived facts, and engine outcomes coexist under this contract. Paper 2 has no mutation, rollback, branch scope, or retraction; those remain Paper 4 concerns.

4 Mixed Benchmark and Conditions

The deterministic smoke generator crosses three engines with derivation depths 1, 2, and 4; distractor counts 0 and 4; and two examples per cell. Arithmetic uses addition chains. Horn tasks ask reachability over link chains using base and recursive rules. Graph tasks ask transitive ISA membership. Six controls contain embedded, quoted, unknown, or non-event forms. This yields 36 engine tasks and six controls.

Table 1: Protocol conditions. Scripted conditions validate compatibility only.

Condition	Available capability	Future empirical role
No engine	None	Unassisted model baseline
Explicit tools	All three, explicitly selected	Conventional invocation baseline
Single calculator	Calculator only	Homogeneous capability control
Single Horn	Horn only	Homogeneous capability control
Single graph	ISA/frame only	Homogeneous capability control
Heterogeneous	Strict automatic router, all engines	Proposed condition
Oracle selection	Gold family, all engines	Routing headroom

Table 2: Scripted smoke results over 36 tasks and six controls per condition. Every row is marked `empirical: false`.

Condition	Task accuracy	Trigger recall	Control false rate
No engine	0.000	0.000	0.000
Explicit tools	1.000	1.000	0.000
Single calculator	0.333	0.333	0.000
Single Horn	0.333	0.333	0.000
Single graph	0.333	0.333	0.000
Heterogeneous	1.000	1.000	0.000
Oracle selection	1.000	1.000	0.000

The smoke harness supplies canonical events rather than model generations. Thus explicit, heterogeneous, and oracle conditions receive equivalent correct requests. A future empirical run must use matched prompts and separately score event exposure, engine selection, argument correctness, result use, and final claims.

5 Non-Empirical Protocol Smoke

The run emits 294 predictions and 1,180 trace records: 216 routing, 144 execution, 784 state-update, and 36 no-event records. Of the state updates, 224 carry non-empty derivation dependencies. Table 2 reports expected plumbing behavior, not experimental support.

The perfect heterogeneous line proves only that a canonical request reaches the correct deterministic engine. Single engines solve exactly their own 12 tasks. At depth four with four distractors, mean scripted Horn runtime is about 5.0 ms, versus 0.085 ms at depth one without distractors. This comparison is confounded by append-only state accumulation and benchmark order and is not a fitted scaling estimate.

6 Future Empirical Protocol

A gate-passing run must use pinned matched checkpoints and multiple seeds. It must cross engine family, derivation depth, entity/fact count, rule count, distractors, state reuse, and model size. Required baselines are no engine, matched prompting, explicit tools, each single engine, heterogeneous automatic routing, and oracle selection. Report final exact accuracy with intervals; detection, normalization, route, engine, persistence, reinjection, and result-use errors; generated and reinjected tokens; model and engine calls; engine work; latency; state bytes/items; reuse rate; and condition-by-depth/model-size interactions. Preserve raw generations and every state transition.

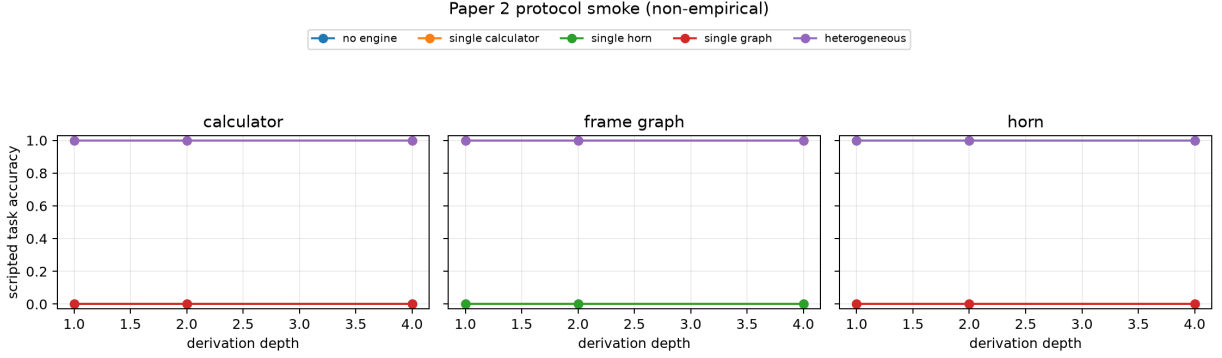


Figure 1: Non-empirical compatibility diagnostic at four distractors. Curves show expected engine availability, not neural reliability or scaling evidence.

7 Falsification and Gate

Heterogeneous assistance is unsupported if explicit tools match or dominate its quality–cost frontier, routing errors erase engine gains, mixed tasks do not benefit beyond their corresponding single engines, persistent state is not reused, state growth costs exceed saved work, or scaling slopes are indistinguishable. The current smoke test cannot satisfy or falsify these criteria. Model experiments remain blocked until Paper 1 identifies a reproducible automatic-assistance regime or a negative result specifically motivates heterogeneous compute.

8 Limitations

The synthetic events reveal the target engine explicitly, eliminating semantic routing. Canonical typed payloads eliminate model formalization errors. The logic and graph tasks are positive queries with simple chains, and the plot cannot estimate generalization. Persistent state grows across a fixed task order, has no eviction, and may create latency confounds. Graph closure is recomputed, not incrementally materialized. There is no constraint engine, natural-language parser, model generation, explicit textual-tool parser, reinjection/use test, XPU experiment, multi-seed variance, or second model.

9 Reproduction

From the repository root, no Docker or accelerator is required:

```
python -m ccpu paper2 generate \
  --config configs/paper2/smoke.json \
  --output artifacts/paper2/smoke/dataset.jsonl
python -m ccpu paper2 validate \
  --dataset artifacts/paper2/smoke/dataset.jsonl
python -m ccpu paper2 simulate \
  --dataset artifacts/paper2/smoke/dataset.jsonl \
  --output-dir artifacts/paper2/smoke/run
python -m ccpu paper2 plot \
  --summary artifacts/paper2/smoke/run/summary.json \
```

```
--output docs/papers/paper2/paper2_protocol_smoke.png
```

The dataset and run manifests hash their inputs and outputs and record repository and environment provenance. Paper-specific code lives under `src/ccpu/paper2`; shared contracts and artifact utilities remain under `src/ccpu/common`.

10 Related Work

Toolformer learns model-authored API use [1]. AlphaGeometry combines neural guidance with symbolic deduction [2]. Dynamic solver composition and verifiable primitive programs study complementary ways to select or compose formal reasoning systems [3, 4]. Paper 2 does not claim heterogeneous tools or neuro-symbolic reasoning as new; it isolates strict automatic routing plus bounded persistent typed state as a falsifiable systems experiment.

11 Conclusion

Paper 2 now has a tested calculator/Horn/graph runtime, bounded persistent state, derivation provenance, mixed benchmark, condition matrix, artifacts, and reproduction path. The implementation is ready for a future preregistered model experiment. It is not evidence that heterogeneous reflex reasoning helps a language model, and the series gate remains closed.

References

- [1] Schick et al. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS, 2023.
- [2] Trinh et al. Solving Olympiad Geometry without Human Demonstrations. Nature, 2024.
- [3] Xu et al. Adaptive LLM-Symbolic Reasoning via Dynamic Logical Solver Composition. EACL, 2026.
- [4] Bhat et al. Forethought: Verifiable Reasoning from Neurosymbolic Primitive Programming. arXiv:2607.04096, 2026.